

ITS4243 - MICROSERVICES AND CLOUD COMPUTING

Faculty of Technology - University of Sri Jayewardenepura

Hansini Gunasena

Visiting Lecturer

Senior Software Engineer – Virtusa Pvt Ltd

BSc In Computer science

T2. MICROSERVICES DESIGN PRINCIPLES – Part 2

/// Concept 6: Asynchronous Communication & Event-Driven Patterns

What Is Asynchronous Communication?

In asynchronous communication, a service **sends a message or event** and continues working **without waiting for an immediate response**.

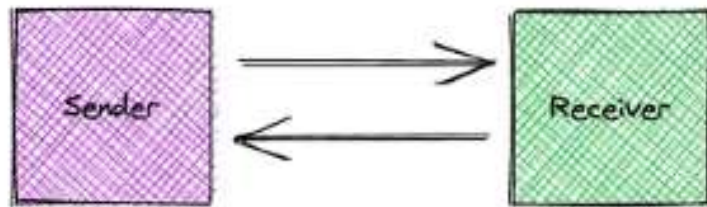
Unlike synchronous REST calls (where one service must wait), asynchronous interactions are **non-blocking, decoupled, and highly scalable**.

Sync vs Async Communication

Request and wait for a response
Fire and forget with messages/events

@boyney123

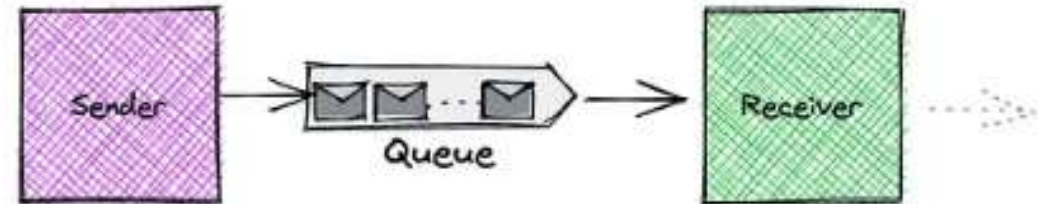
Synchronous



Request-response model

Send a request and wait for some response
Example: API requests

Asynchronous



Fire and forget model

Send message/event and forget
Example: Event Driven Architecture

◆ Synchronous vs. Asynchronous

Aspect	Synchronous	Asynchronous
Communication	Request → Response	Message/Event based
Coupling	Tight	Loose
Availability	Dependent on receiver	Independent
Example	REST call (Order-Service → Payment-Service)	Event publish (OrderCreated event)
Reliability	Fails if receiver is down	Message can be queued or retried

Why Asynchronous Design Matters

- **Improves performance:** No waiting for dependent services
- **Enhances reliability:** Temporary failures are absorbed via queues
- **Scales easily:** Consumers can be added dynamically
- **Supports real-time event flow:** Enables reactive systems
- **Encourages autonomy:** Services evolve independently

Common Asynchronous Communication Patterns

1. Event-Driven Architecture (EDA)

- Services publish **events** when something significant happens (e.g., “OrderCreated”).
- Other services **subscribe** to react to those events.
- Promotes *eventual consistency* and *loose coupling*.

Event-Driven Architecture (EDA) is a software design paradigm where system components communicate by producing and responding to events. These events can be important happenings, like user actions or changes in the system's state.

<https://www.geeksforgeeks.org/system-design/event-driven-architecture-system-design/>

Example:

sql

Order-Service publishes "OrderCreated"

↓

Inventory-Service updates stock

↓

Email-Service sends confirmation

↓

Analytics-Service logs order data

2. Message Queue Pattern

- Uses a **message broker** (like RabbitMQ or ActiveMQ).
- Producers send messages; consumers process them asynchronously.
- Ensures reliable delivery and load balancing.

Example:

`Notification-Service` queues messages for `Email-Service` and `SMS-Service` to process independently.

3. Event Streaming

- Continuous event flow with **real-time processing**.
- Implemented using **Apache Kafka, Pulsar, or Google Pub/Sub**.
- Ideal for analytics, IoT data, or transaction pipelines.
- **Example:**
Netflix uses **Kafka** for real-time tracking of user actions → feeds into recommendation systems.

4. Event Sourcing

- Instead of storing current state, services store **all state changes as events**.
- The system's state can be reconstructed by replaying those events.
- Perfect for audit logs and temporal analytics.
- All stored as events → replayable for analytics or recovery.

nginx

PaymentInitiated

PaymentProcessed

PaymentCompleted

◆ Key Technologies

Category	Tools
Message Broker	RabbitMQ, ActiveMQ
Event Streaming	Apache Kafka, Redpanda, Google Pub/Sub
Event Processing	Apache Flink, Spark Streaming, KSQL
Event Tracing	OpenTelemetry, Jaeger
Integration	Spring Cloud Stream, Kafka Streams, Debezium

◆ Example: Event-Driven Order Workflow

Scenario:

E-commerce microservice interaction via asynchronous events.

markdown

1. Order-Service publishes "OrderCreated"
2. Payment-Service subscribes → processes payment
3. Inventory-Service subscribes → updates stock
4. Notification-Service subscribes → sends confirmation email

Advantages:

- No single point of failure
- Services can scale independently
- Events processed at different speeds

◆ Best Practices for Implementing Asynchronous Communication

✓ Design clear event schemas

- Define standard formats (JSON, Avro, Protocol Buffers).
- Include metadata: event type, timestamp, trace ID.

✓ Use idempotency

- Repeated event processing should not cause duplicate actions.
(e.g., same “OrderCreated” event shouldn’t charge customer twice.)

✓ Ensure observability

- Implement tracing across event chains using correlation IDs.

✓ Manage dead-letter queues (DLQs)

- Handle undeliverable or failed events safely.

✓ Handle versioning

- Maintain backward compatibility for event consumers.

✓ Use event choreography or orchestration wisely

- **Choreography:** Services react to each other’s events (decentralized).
- **Orchestration:** A central service (or Saga) coordinates workflow.

Real-World Examples

- **Netflix:** Uses Kafka for 1+ trillion events per day for recommendations and monitoring.
- **Uber:** Real-time trip updates and surge pricing use event-driven design.
- **Amazon:** Orders, shipments, and inventory all interact via asynchronous events.

◆ Summary Table

Concept	Description	Example Tool
Event-Driven Architecture	Services communicate via events	Kafka, RabbitMQ
Message Queue	Buffered asynchronous delivery	ActiveMQ, SQS
Event Streaming	Real-time event pipelines	Kafka, Pulsar
Event Sourcing	State stored as events	EventStoreDB
Choreography	Services react to events	Spring Cloud Stream
Orchestration	Centralized workflow	Temporal.io, Camunda

Concept 7 : Observability (Logging, Metrics, Tracing)

What Is Observability?

is the ability to understand what's happening **inside a distributed system** based on the data it produces — logs, metrics, and traces.

It helps you **monitor**, **debug**, and **optimize** microservices in production environments.

🔑 Key Idea

Traditional monitoring answers *“Is my service up?”*

Observability answers *“Why is it behaving this way?”*

Why Observability Matters in Microservices

Challenge	Why It Matters
Many independent services	Each service generates its own data; we need a unified view
Dynamic scaling (Kubernetes, Docker)	Instances come and go; logs alone aren't enough
Asynchronous & event-driven flows	Harder to trace request paths
Distributed ownership	Teams need to find root causes quickly
Goal: Detect, diagnose, and resolve issues <i>before</i> users notice them.	

The Three Pillars of Observability

Logging

- Logs are **structured records of events** in your system.
- **Purpose:** Capture details of what happened and when.
- **Types:**
 - **Application logs:** Info, warning, error logs
 - **Access logs:** Who called which endpoint
 - **Audit logs:** Security or compliance-related actions
- **Best Practices**
 - Use structured formats (JSON instead of plain text).
 - Include correlation IDs (trace ID, span ID, request ID).
 - Centralize logs using **ELK Stack (Elasticsearch + Logstash + Kibana)** or **Grafana Loki**.

Example (JSON log entry):

json

```
{  
  "timestamp": "2025-10-19T08:15:30Z",  
  "service": "order-service",  
  "level": "ERROR",  
  "message": "Payment timeout",  
  "traceId": "abc123xyz"  
}
```

Tools:

•ELK Stack, Fluentd, Grafana Loki, Cloud Logging, AWS
CloudWatch

b. Metrics

numeric representations of system performance over time.

Examples:

- CPU utilization, memory usage
- Request latency, error rates, throughput
- Custom metrics: “orders_processed_per_minute”

Purpose:

Track trends, detect anomalies, and trigger alerts.

Best Practices

- Define SLIs/SLOs (Service Level Indicators/Objectives)
- Aggregate at service and system level
- Expose `/actuator/metrics` in Spring Boot

Tools:

- **Prometheus** for collection
- **Grafana** for visualization
- **Micrometer** as Java metric facade
- **Cloud-native metrics** (GCP Monitoring, AWS CloudWatch)

Example Alert Rule (Prometheus):

cpp

```
ALERT HighErrorRate
```

```
  IF rate(http_requests_total{status="500"}[5m]) > 0.05
```

```
  FOR 2m
```

```
  LABELS { severity="critical" }
```


c. Tracing

- Traces show the **journey of a single request** across multiple microservices.
- **Purpose:** Identify latency bottlenecks and failure points in distributed systems.

Example Flow:

arduino

Client → API Gateway → Order-Service → Payment-Service → Email-Service

If total response time is 5s, tracing helps identify *which hop* caused delay.

Key Concepts:

- **Trace ID:** Unique identifier for one request flow
- **Span:** A single operation within a trace
- **Parent/Child Relationship:** Span hierarchy forms a tree or timeline

Tools:

- **Jaeger, Zipkin, OpenTelemetry, Honeycomb**
- Integrated with frameworks like **Spring Cloud Sleuth**

Example (Trace Timeline Visualization):

CSS

Order-Service [200ms]

└─ Payment-Service [2.5s] ← Problem area

└─ External API [2.3s]

Concept 8: Resiliency & Fault Tolerance in Microservices

What is Resiliency?

is the ability of a system to **recover gracefully** and continue functioning even when parts of it fail. Failures are **inevitable** in distributed systems — networks go down, services crash, timeouts occur — but a resilient system ensures that a small issue doesn't cause a **domino effect**.

◆ 2 Why It's Important in Microservices

Challenge	Impact Without Resilience	Example
Service dependency failure	Request chain breaks	Payment service down → entire checkout fails
Network latency	User experience degrades	Timeout on inventory lookup
Instance crash	Data loss or errors	Crash in 1 pod affects orders
Traffic spikes	System overloads	Flash sale causing cascading failures

Goal: Keep services responsive and stable under failure conditions.

Difference: Resiliency vs Fault Tolerance

Concept	Definition	Example
Resiliency	System adapts and recovers from failure	Retry, fallback, or reroute requests
Fault Tolerance	System continues operating even when a fault occurs	Replication, failover, redundancy

🧠 *Resiliency = Software techniques* (retries, circuit breakers)

⚙️ *Fault Tolerance = Infrastructure-level techniques* (replication, clustering)

Common Resilience Patterns

a. Circuit Breaker Pattern

- Detects repeated failures and stops calls to the failing service temporarily.
- Prevents cascading failures and allows recovery time.

Analogy: Like a home electrical circuit breaker — if one appliance trips, others keep working.

Example:

If `Payment-Service` is slow/unavailable, the `Order-Service` “opens the circuit” and immediately fails fast instead of waiting for timeouts.

Tools:

- **Resilience4j** (modern alternative to Hystrix)
- **Spring Cloud Circuit Breaker**

Diagram:

mathematica

Order-Service → Payment-Service (fails)

Circuit opens → Fallback triggered

After timeout → Half-open → Retest → Closed if OK

- **b. Retry Pattern**

- Automatically retries failed operations a set number of times before giving up.
- Useful for transient errors like network glitches.

Example:

yaml

```
resilience4j.retry:  
  instances:  
    paymentRetry:  
      max-attempts: 3  
      wait-duration: 2s
```

Best Practices:

- Use exponential backoff (2s → 4s → 8s)
- Combine with circuit breakers to avoid retry storms

c. Fallback Pattern

- Provide an **alternative response** or **cached data** when a service is unavailable.

Example:

If **Inventory-Service** is down → Return last known stock level or “service unavailable” message gracefully.

Goal: Degrade functionality, not user experience.

Fault Tolerance Mechanisms (Infra-level)

Mechanism	Description
Load Balancing	Distribute requests evenly (NGINX, Kubernetes)
Replication	Multiple service instances handle traffic
Failover	Automatic switch to a backup system
Redundancy	Duplicate critical components for reliability
Auto-scaling	Spin up/down instances during load changes
Chaos Testing	Inject failures to test resilience (e.g., Chaos Monkey)

Best Practices

- ✓ Design for failure — assume dependencies *will* fail.
- ✓ Combine multiple patterns — e.g., **Retry + Circuit Breaker + Fallback**.
- ✓ Don't retry indefinitely — use exponential backoff.
- ✓ Implement **timeouts** for all external calls.
- ✓ Test resilience regularly using **chaos experiments**.
- ✓ Ensure **idempotency** — repeated retries shouldn't duplicate actions.
- ✓ Monitor circuit states and failure trends continuously.

Concept 9 : Automation (CI/CD)

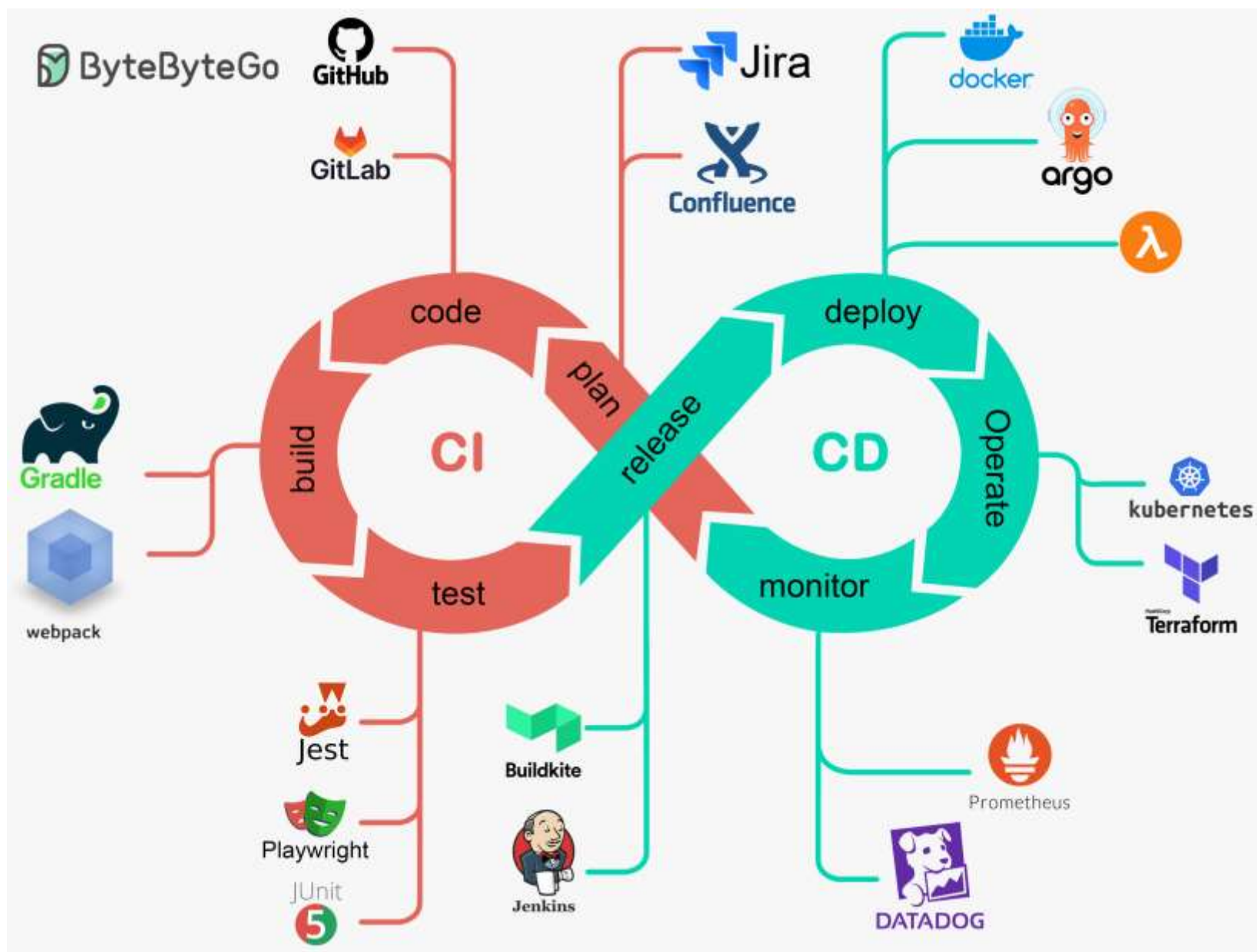
- Definition: Automate code building, testing, and deployment for faster delivery.
- Pipeline Stages:
 - 1. Continuous Integration (CI): auto build & test (Jenkins, GitHub Actions)
 - 2. Continuous Delivery (CD): deploy to staging automatically
 - 3. Continuous Deployment: promote to production with approvals

What is CI/CD?

CI/CD (Continuous Integration and Continuous Deployment/Delivery)

automates the **building, testing, and deployment** of software, ensuring that changes move from code → production safely and efficiently.

Automation is essential for **microservices**, which often have dozens or hundreds of small, independently deployed components.



- Thank you.....